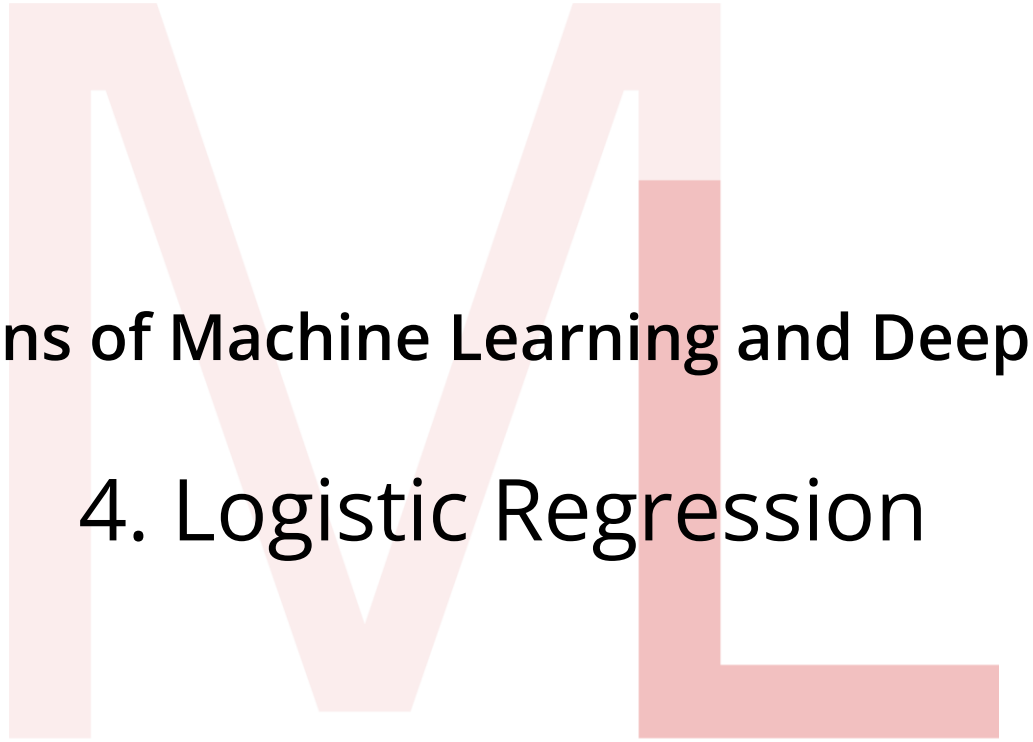
Foundations of Machine Learning and Deep Learning

4. Logistic Regression



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

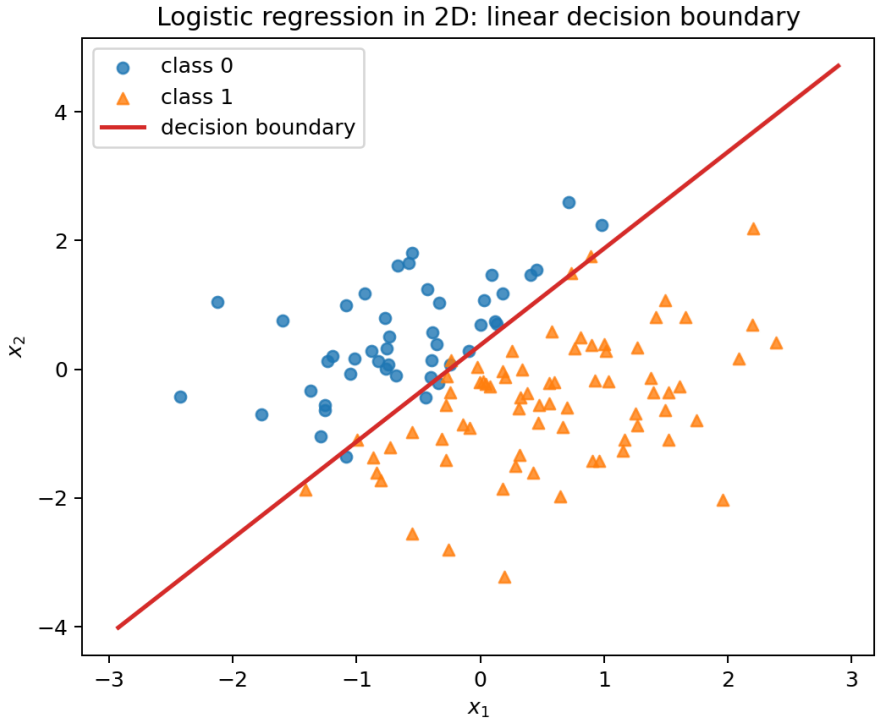
Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Gradient descent
- Polynomial features
- Regularization
- Implementation

Introduction

- **Logistic regression** is a binary classification parametric model
- Predicts probabilities and outputs in $[0,1]$

- Same cycle as in Linear Regression: model → loss → optimization



Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Gradient descent
- Polynomial features
- Regularization
- Implementation

Model

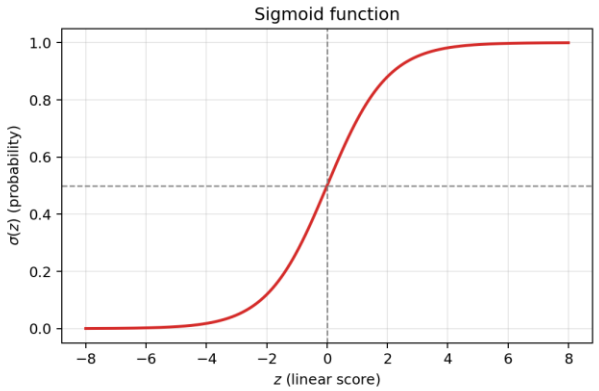
- **Main idea:**
  - We want to learn a function that maps input features **x** to a probability, $\hat{y} \in (0,1)$, indicating that the input belongs to class 1
- **Model assumption**

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where *z* is the linear score (logit*)

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

σ = sigmoid function, maps any real number into (0,1)



  - Each feature x_j contributes proportionally via its weight w_j
  - The bias b shifts the decision threshold
  - The prediction $\hat{y}$ is interpreted as $\hat{y} \approx P(y = 1 \mid \mathbf{x})$

Model

- **Model assumption**
  - *What is a Logit?
    - The logit function is the inverse of the sigmoid:

$$p = \sigma(z) \quad \Longleftrightarrow \quad z = \text{logit}(p)$$

    - It is defined as

$$\text{logit}(p) = \log \frac{p}{1-p}$$

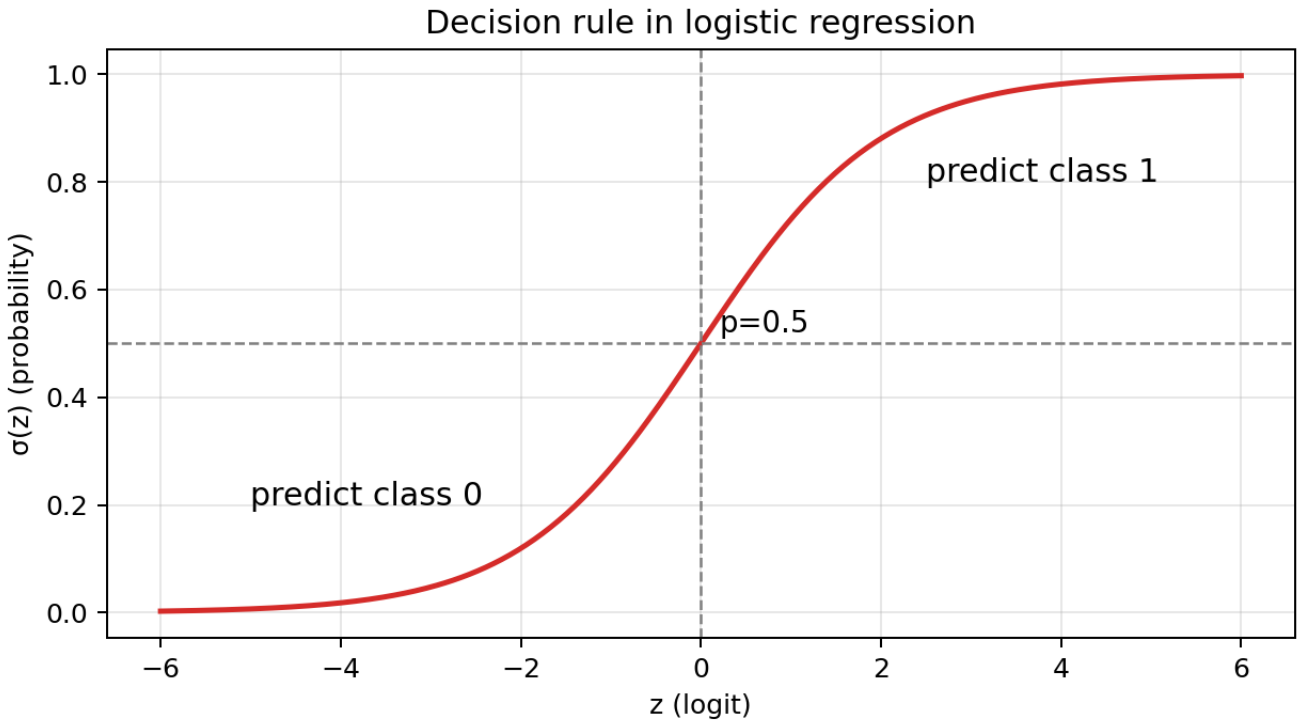
    - Since $\frac{p}{1-p} = \frac{P(y=1 \mid \mathbf{x})}{P(y=0 \mid \mathbf{x})}$ is the odds, the logit is the log-odds
    - Logistic regression assumes the **log-odds are linear in the features**

Model

- **Decision rule**

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

  - If $\hat{y} \geq 0.5$, predict class 1
    - Equivalently, if $z \geq 0$
  - If $\hat{y} < 0.5$, predict class 0
    - Equivalently, if $z < 0$



  - The decision boundary is where $\hat{y} = 0.5$ (i.e., where $z = 0.5$)
  - This corresponds to the hyperplane $\{x : x^\top w + b = 0\}$

Model

Example

- Suppose have two features and a fitted logistic regression model

$$z = 2.0 \cdot x_1 - 1.0 \cdot x_2 + 0.5$$

- For a new instance:
  - $x_1 = 1.0, x_2 = 2.0$
  - Linear score (logit): $z = 2(1.0) - 1(2.0) + 0.5 = 0.5$
  - Sigmoid: $\hat{y} = \sigma(0.5) \approx 0.62$
  - Decision: Since $\hat{y} \geq 0.5$, predict class 1

Model

- **Model in vector form (single example)**

$$z^{(i)} = \mathbf{x}^{(i)\top}\mathbf{w} + b, \quad \hat{y}^{(i)} = \sigma(z^{(i)})$$

where

  - $\mathbf{x}^{(i)} \in \mathbb{R}^d$ is the feature vector of instance *i*
  - $\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}$
  - $\hat{y}^{(i)}$ is the predicted probability of class 1

- **Model in matrix form (dataset)**

$$\mathbf{Z} = X\mathbf{w} + b\mathbf{1}, \quad \hat{\mathbf{y}} = \sigma(\mathbf{Z})$$

where

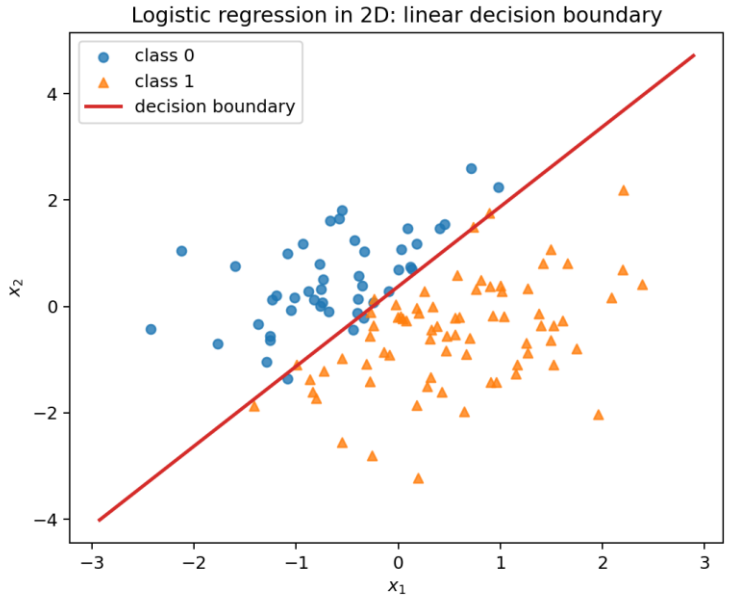
  - $X \in \mathbb{R}^{n \times d}$
  - $\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}, \mathbf{1} \in \mathbb{R}^n$
  - $\hat{\mathbf{y}} \in (0,1)^n$ is the vector of predicted probabilities

Model

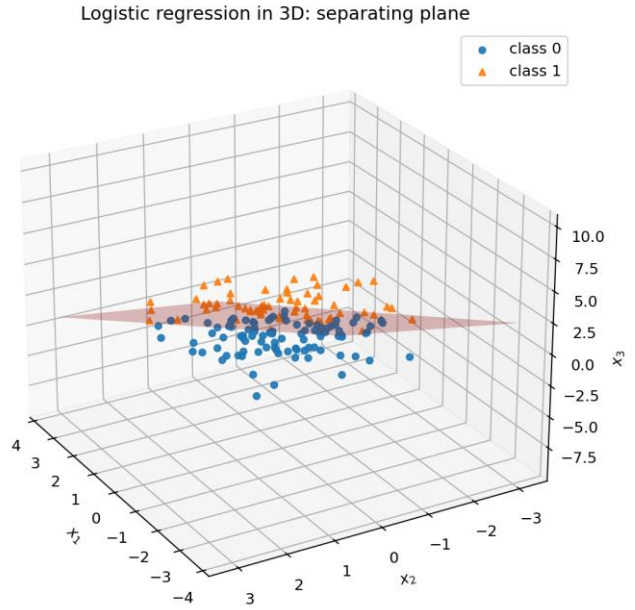
- **Geometric interpretation**
  - Finding a *d*-dimensional hyperplane (boundary) embedded in $\mathbb{R}^{d+1}$

$$\{x : x^\top w + b = 0\}$$

For 2D: linear boundary
$d = 2$



For 3D: boundary plane
$d = 3$

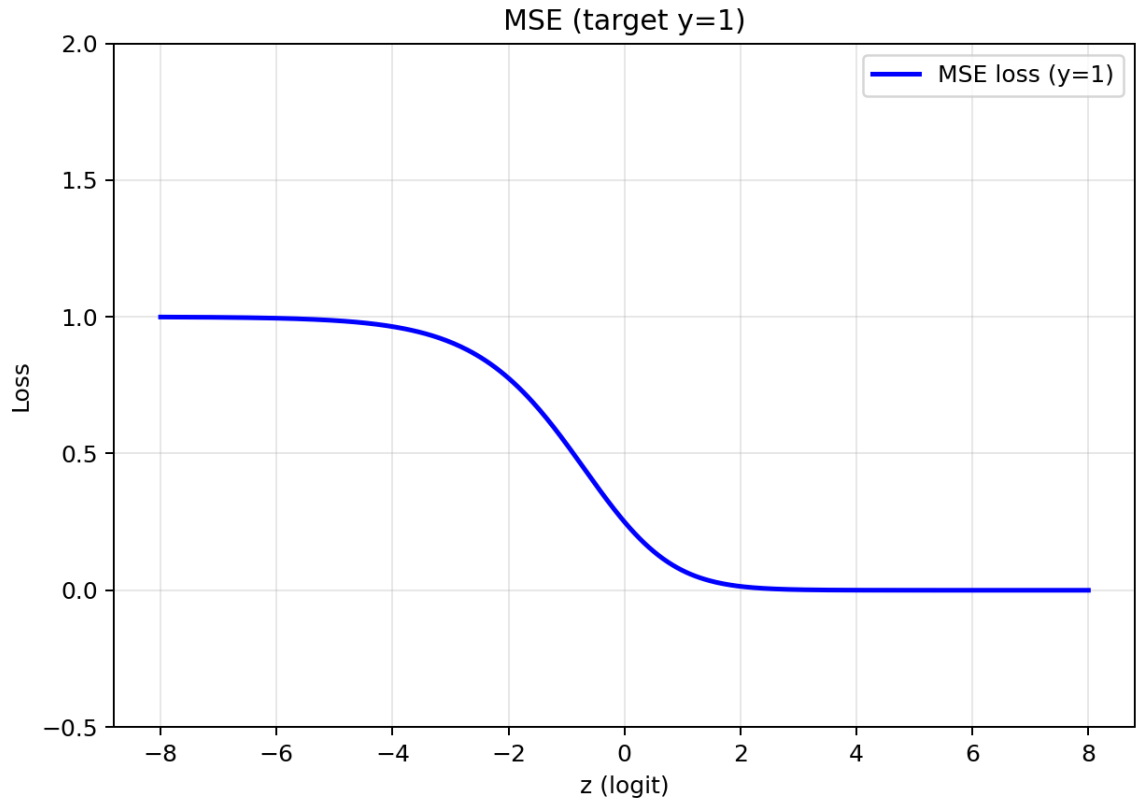


Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Gradient descent
- Polynomial features
- Regularization
- Implementation

Loss function

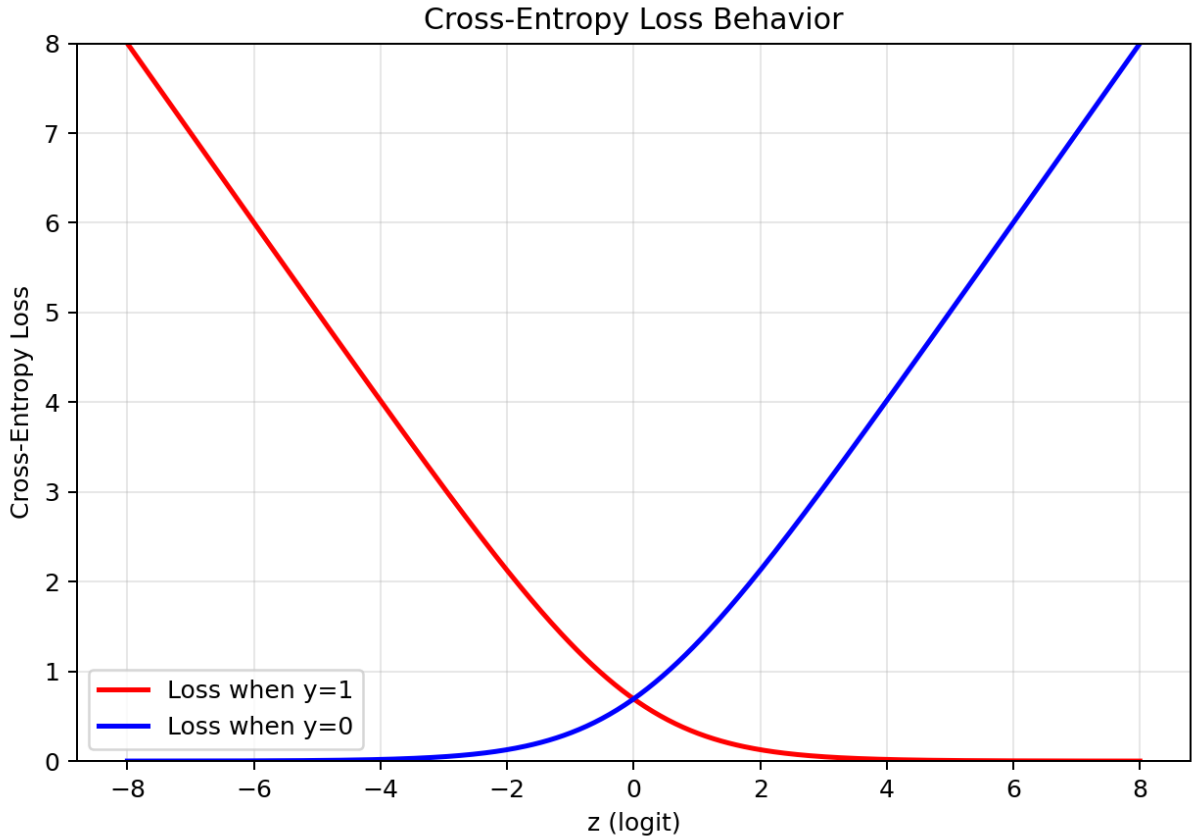
- **Why MSE is not ideal**
  - If we try to use MSE with sigmoid outputs:
    - Loss surface is non-convex
    - Flat surfaces (gradients vanish)
  - We need a better loss



Loss function

- **Cross entropy**
  - For a single sample i
    - If $y = 1$, then
      $$loss = -\log(\hat{y})$$
    - If $y = 0$, then
      $$loss = -\log(1 - \hat{y})$$
  - Wrong predictions → big penalty
  - Correct predictions → tiny penalty

$$\mathcal{L}(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$



Loss function

- **Cross entropy loss**
  - For a dataset (*n* samples)

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} -\left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

    - It is convex in the parameters (no local minima)
    - Differentiable, well-suited for gradient descent
    - Matches probability interpretation for logistic regression

Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Gradient descent
- Polynomial features
- Regularization
- Implementation

Probabilistic view

- **Assumption**
  - Each label $y^{(i)} \in \{0, 1\}$ is a Bernoulli random variable with success prob. $\hat{y}^{(i)}$

$$P(y^{(i)} \mid x^{(i)}) = \left(\hat{y}^{(i)}\right)^{y^{(i)}} \left(1 - \hat{y}^{(i)}\right)^{1-y^{(i)}}$$

  - where $\hat{y}^{(i)} = \sigma\left(x^{(i)\top} w + b\right)$

The Bernoulli distribution models a binary outcome:

$$y \in \{0, 1\}, \quad P(y = 1) = p, \quad P(y = 0) = 1 - p$$

And the probability mass function is:

$$P(y \mid p) = p^y (1 - p)^{1-y}, \quad 0 \leq p \leq 1$$

Examples: coin flip, spam vs. not spam, pass vs. fail... (p = probability of success)

Probabilistic view

- **Likelihood of the dataset**

The likelihood measures how plausible the observed data is, for a given set of parameters

$$L(\mathbf{w}, b, \sigma^2 \mid X, \mathbf{y}) = p(\mathbf{y} \mid X, \mathbf{w}, b, \sigma^2)$$

$$p(\mathbf{y} \mid X, \mathbf{w}, b, \sigma^2) = \prod_{i=1}^{n} P(y^{(i)} \mid x^{(i)}) = \prod_{i=1}^{n} \left(\hat{y}^{(i)}\right)^{y^{(i)}} \left(1 - \hat{y}^{(i)}\right)^{1-y^{(i)}}$$

Maximizing the likelihood is equivalent to maximizing the log-likelihood

Probabilistic view

- **Log-Likelihood of the dataset**

$$p(\mathbf{y} \mid X, \mathbf{w}, b, \sigma^2) = \prod_{i=1}^{n} P(y^{(i)} \mid x^{(i)}) = \prod_{i=1}^{n} \left(\hat{y}^{(i)}\right)^{y^{(i)}} \left(1 - \hat{y}^{(i)}\right)^{1-y^{(i)}}$$

$$\ell(w, b) = \sum_{i=1}^{n} \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

$$\mathcal{L}(w, b) = -\ell(w, b)$$

Maximizing the likelihood is equivalent to minimizing the cross entropy loss

$$-\sum_{i=1}^{n} \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Probabilistic view

- **Conclusion**
  - The cross entropy loss is not arbitrary
    - It is the negative of the log likelihood of the probabilistic model
  - Logistic regression = maximum likelihood estimation for a Bernoulli model

Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Gradient descent
- Polynomial features
- Regularization
- Implementation

How are the parameters learnt? – Gradient descent

- **Gradient descent**
  - There is no closed-form solution for the logistic regression loss
  - We must use iterative optimization (gradient descent)

$$\mathcal{L}(w,b) = -\sum_{i=1}^{n} \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

with $\hat{y}^{(i)} = \sigma(x^{(i)\top} w + b)$



$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{n} \sum_{i=1}^{n} \left(\hat{y}^{(i)} - y^{(i)} \right) x^{(i)}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} \left(\hat{y}^{(i)} - y^{(i)} \right)$$

How are the parameters learnt? – Gradient descent

- **Gradient descent for logistic regression**
  - Update each weight:

$$w_j \leftarrow w_j - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_j}, \quad j = 1, \ldots, d$$

  - Update bias:

$$b \leftarrow b - \eta \cdot \frac{\partial \mathcal{L}}{\partial b}$$

  - Repeat until convergence (or max iters reached)

How are the parameters learnt? – Gradient descent

- **Note: The importance of scaling**
  - The scale of the features has a big impact on the gradient
  - Larger scale features dominate updates
    - Slower convergence (zig-zag)
    - Numerical instability
  - Standard approach: standardization $x_j \longmapsto \frac{x_j - \mu_j}{\sigma_j}$

Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Polynomial features
- Regularization
- Implementation

Polynomial features

- **What if classes are not linearly separable?**
  - Some classification problems need nonlinear decision boundaries
  - But Logistic regression produces a linear separation

  - Solution: expand the feature space with polynomial terms

  - **Example**: For two features (x_1, x_2)
    - Original (linear):

$$z = w_1 x_1 + w_2 x_2 + b$$

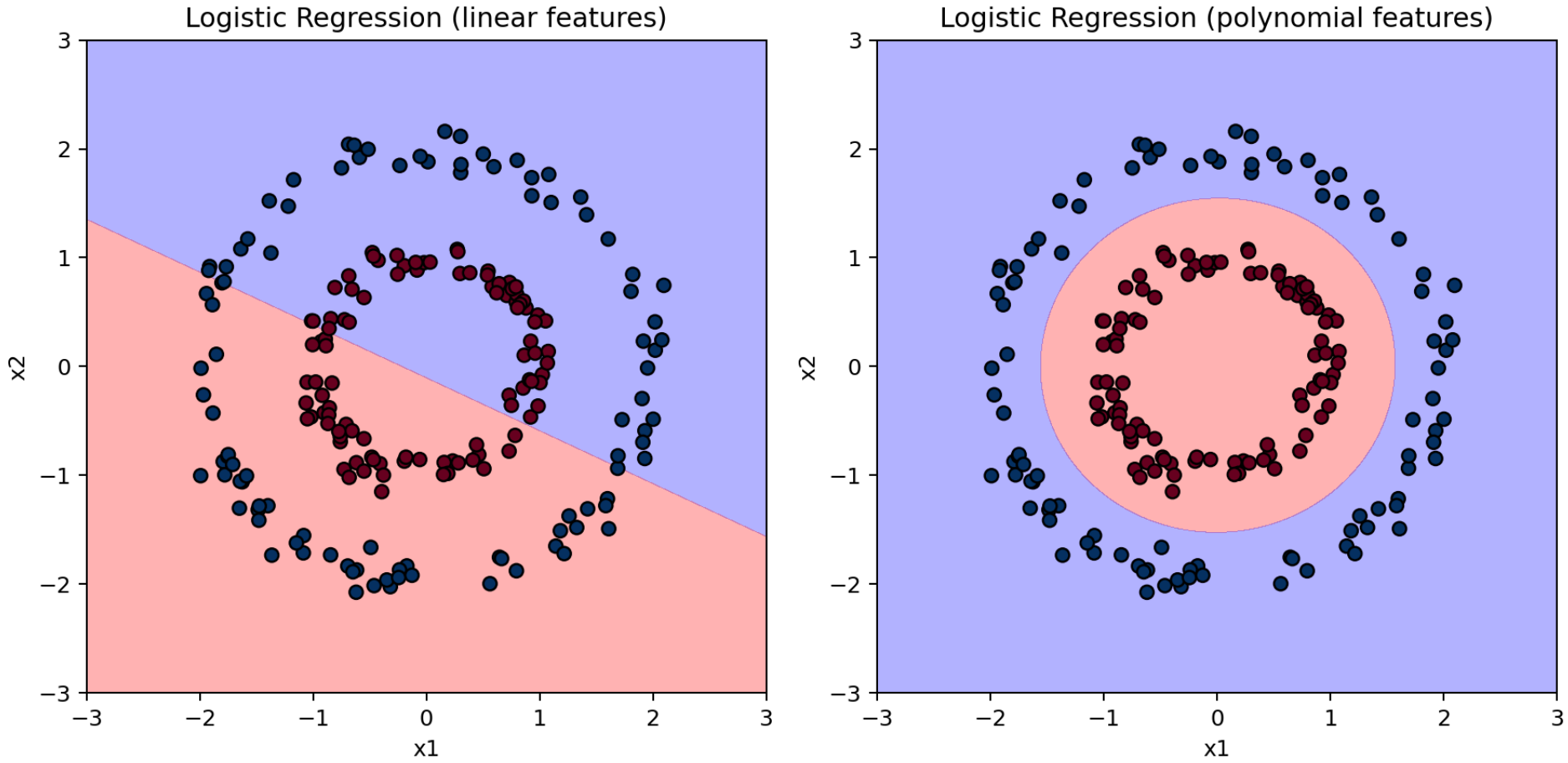
    - With degree-2 polynomial features:

$$z = w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2 + w_5 x_1 x_2 + b$$

Decision boundary is now a conic curve

Polynomial features

- **Decision boundaries with polynomial features**



Polynomial features

- **Pros of polynomial features**
  - Easy to implement with feature engineering
  - Can capture nonlinear relationships
  - They are also valid for linear regression

- **Cons of polynomial features**
  - Feature space grows quickly (risk of overfitting)
  - Scaling becomes even more important
  - Still limited (the nonlinearities are fixed) – no so with neural networks

Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Polynomial features
- Regularization
- Implementation

Regularization

- **Idea**
  - Adding a penalty term to the loss to keep weights small
  - Intuition: small weights = simpler model = better generalization

$$\mathcal{L}(w,b) = -\sum_{i=1}^{n}\left[y^{(i)}\log\hat{y}^{(i)} + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right] + \lambda\Omega(w)$$

Regularization strength: controls the bias variance tradeoff

- L^2 regularization $\quad \Omega(w) = \frac{1}{2}\|w\|_2^2 = \frac{1}{2}\sum_j w_j^2$
- L^1 regularization $\quad \Omega(w) = \|w\|_1 = \sum_j |w_j|$

Also valid for linear regression

Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Polynomial features
- Regularization
- Implementation

Implementation – scikit-learn

- **Scikit-learn** (Usual mechanics: .fit(), .predict()...)
  - LogisticRegression:
    - Standard implementation (includes L^1 and L^2 regularizations)
  - SGDClassifier:
    - Optimizes logistic regression with stochastic gradient descent

Implementation – PyTorch

- **PyTorch**
  - Equivalent training loop as in linear regression
    - Data setup
    - Initialize parameters
    - Loop:
      - Forward pass (make predictions)
      - Compute loss
      - Backward pass (gradients)
      - Update parameters